

CSE306 – Project 2

A Free-Surface 2D Fluid Solver

Emeric Payer

June 16, 2026

1 Overview

This report describes a free-surface 2D fluid solver, meaning that the fluid has a moving boundary with the surrounding air, just like water poured into a glass.

The idea to enforce incompressibility is the following: instead of tracking the fluid as a continuous field, we represent it as a cloud of particles, and we force each particle to own a small region of fixed area at every instant. Those regions are the cells of a power diagram, choosing them is an optimal transport problem, and a spring then pulls each particle towards the centre of its region. The whole project is therefore three building blocks:

1. Voronoï diagrams, via Voronoï Parallel Linear Enumeration and Sutherland–Hodgman polygon clipping
2. power (Laguerre) diagrams, with their weights optimised by L-BFGS so that each cell reaches a prescribed area
3. the de Gallouet–Mérigot incompressible Euler scheme, which puts the above in motion

All timings reported were measured on my 12th Gen Intel(R) Core(TM) i7-12700H processor, using `g++ -O3 -std=c++11 -fopenmp -I. main.cpp lbfgs.c -o main` then `./main`.

To generate the video simulation, the individual frames were compiled using FFmpeg at a frame rate of 24 fps, encoded with the H.264 codec, and saved in the MP4 format under `output.mp4`.

The values below are the ones used for the final video.

Parameter	Variable	Value
Number of fluid particles	<code>N_particles</code>	300
Fluid volume (fraction of the box)	<code>fluid_volume</code>	0.2
Spring length scale	ε	0.004
Spring stiffness	$1/\varepsilon^2$	6.25×10^4
Time step	<code>dt</code>	0.002
Particle mass	<code>m_i</code>	200
Gravity	g	9.81
Disk discretisation	<code>disk_sides</code>	60
Initial relaxation steps	–	5
Frames simulated	–	1000
Wall restitution (left/right)	–	0.5

Table 1: Parameters used for the final simulation

2 Lab 6 (Voronoi diagrams and clipping)

A Voronoi diagram cuts the plane into regions, one per site, where a region is the set of all points closer to its site than to any other. We build one cell at a time: start from the whole unit square, and for every other site, slice away the half of the cell that is closer to that other site. After slicing against all sites, what remains is the Voronoi cell. Each slice is a polygon-clipping operation performed by Sutherland–Hodgman.

1. **Sutherland–Hodgman clipping** (`clip_by_bisector`). This clips a convex polygon by a single line. We walk the polygon edge by edge (A, B) , and for each edge output the crossing point when the edge straddles the line plus the endpoint B whenever it is inside. Because every clip is a half-plane, the cell stays a single convex polygon throughout.
2. **Parallel Linear Enumeration** (`compute`). The diagram is then just: for each site, clip the square against every other site. We skip using a k -d tree to restrict the comparison to nearby sites, so this is the brute-force $\mathcal{O}(n^2)$ version. Its one virtue is that every cell is computed independently of the others – something we exploit conceptually but not yet in code.
3. **Polygon area, centroid and integral**. Three small geometric routines useful for later:
 - (a) `area` is the shoelace formula
 - (b) `centroid` (used by the spring force) and `integral_square_distance` (used by the transport objective) both split the polygon into a triangle fan from vertex 0 and sum a closed-form per-triangle expression

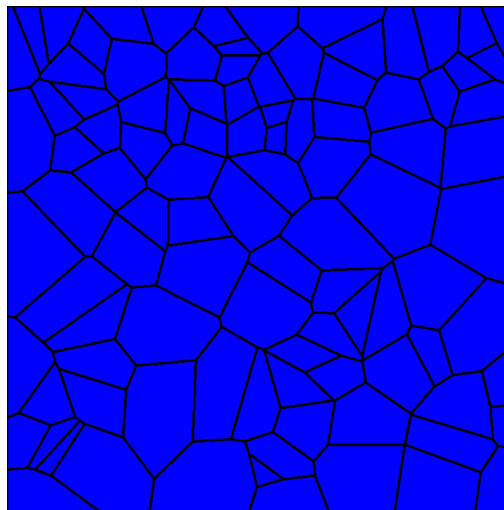


Figure 1: A Voronoi diagram of 100 random sites

3 Lab 7 (Power diagrams and optimal transport)

A power diagram is a Voronoi diagram with a knob on every cell. Each site gets a weight; raising a site’s weight pushes its boundaries outward, so its cell grows at the expense of its neighbours. We want this knob because the fluid solver needs every cell to have a specific area, and the weights serve as the degrees of freedom that let us dial the areas in.

Finding the weights that hit a set of target areas is a semi-discrete optimal transport problem solved with L-BFGS.

1. **Voronoi** $\rightarrow$ **Laguerre**. We reuse the same clipper and only move the dividing line: a point now belongs to site i when its power distance is smallest,

$$\|x - P_i\|^2 - w_i \leq \|x - P_j\|^2 - w_j$$

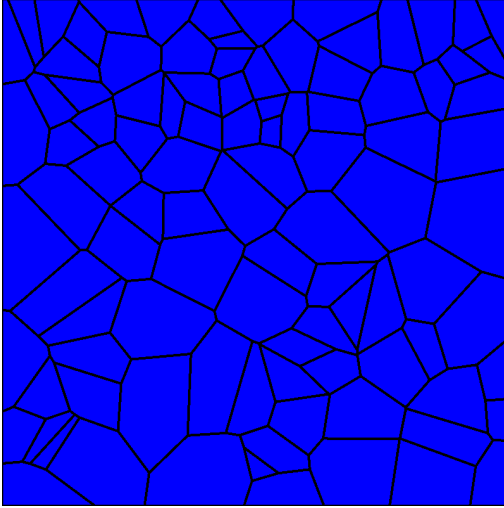
Notice that setting all weights to zero gives back the ordinary Voronoi diagram

2. **L-BFGS**. We want each cell to reach a target area λ_i . The gradient that `evaluate` returns to the optimiser is simply

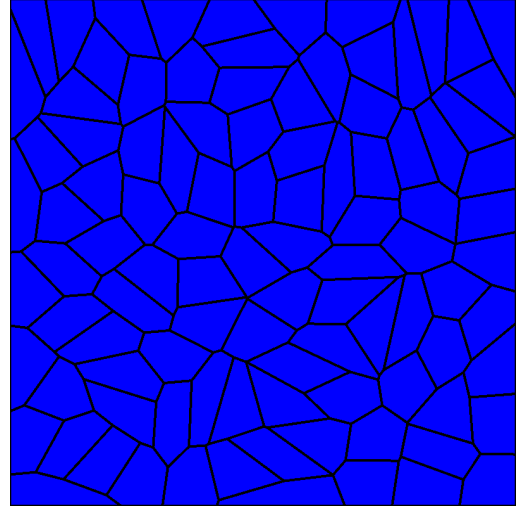
$$g_i = |\text{cell}_i| - \lambda_i$$

i.e. a weight is raised when its cell is too small and lowered when it is too big – the optimiser inflates and deflates cells until the areas match

3. **Result**. The below figure shows that the raw Voronoi diagram has wildly uneven cells (left), while after L-BFGS with equal target areas the cells are visibly equalised (right)



(a) Voronoi: uneven cell areas



(b) Power diagram after L-BFGS

Figure 2: Optimising the weights drives every cell towards the same area

4 Lab 8 (The incompressible Euler scheme)

Now the goal is to set the fluid in motion. At each instant we want every particle to occupy a region of fixed area, and we want those regions to cover only the fluid, not the empty air above it. This is a partial optimal transport problem; one extra “air” weight handles the empty space. Once the cells are known, the physics is simple: gravity pulls the particles down and a spring pulls each particle towards the centre of its own cell.

1. **The air variable**. We append one extra weight w_{air} . Each fluid cell shares the target area $\lambda = \text{fluid_volume}/N$, and the air weight carries the term that pins the total fluid area, with gradient

$$g_{\text{air}} = \text{fluid_volume} - \sum_i |\text{cell}_i|$$

2. **Clipping by a disk**. After the power clipping, each cell is also clipped against a disk centred on its particle of radius

$$r_i = \sqrt{w_i - w_{\text{air}}}$$

(an empty cell when $w_i \leq w_{\text{air}}$), which keeps a cell from stretching into a thin sliver and is what gives a loose particle its round look; when the fluid is squeezed the disk no longer binds and the cell becomes polygonal (we approximate the disk by a 60-gon and clip its edge by edge with `clip_by_edge`)

3. **Forces and integration.** For each particle we add gravity and the Gallouet–Mérigot spring towards the cell centroid c_i

$$F_i = m g + \frac{1}{\varepsilon^2} (c_i - x_i)$$

and integrate with a semi-implicit Euler step (velocity first, then position) which is stable and conserves energy far better than plain explicit Euler

4. **Walls.** Left/right walls bounce the particle back with half its horizontal speed (restitution 0.5); floor/ceiling clamp it and zero the vertical velocity, so no energy is injected at the top and bottom
5. **Starting state.** Particles are seeded by rejection sampling inside a disk near the top of the box, then five Lloyd relaxation steps (snap each particle to its cell centroid) spread the initial blob evenly before gravity takes over

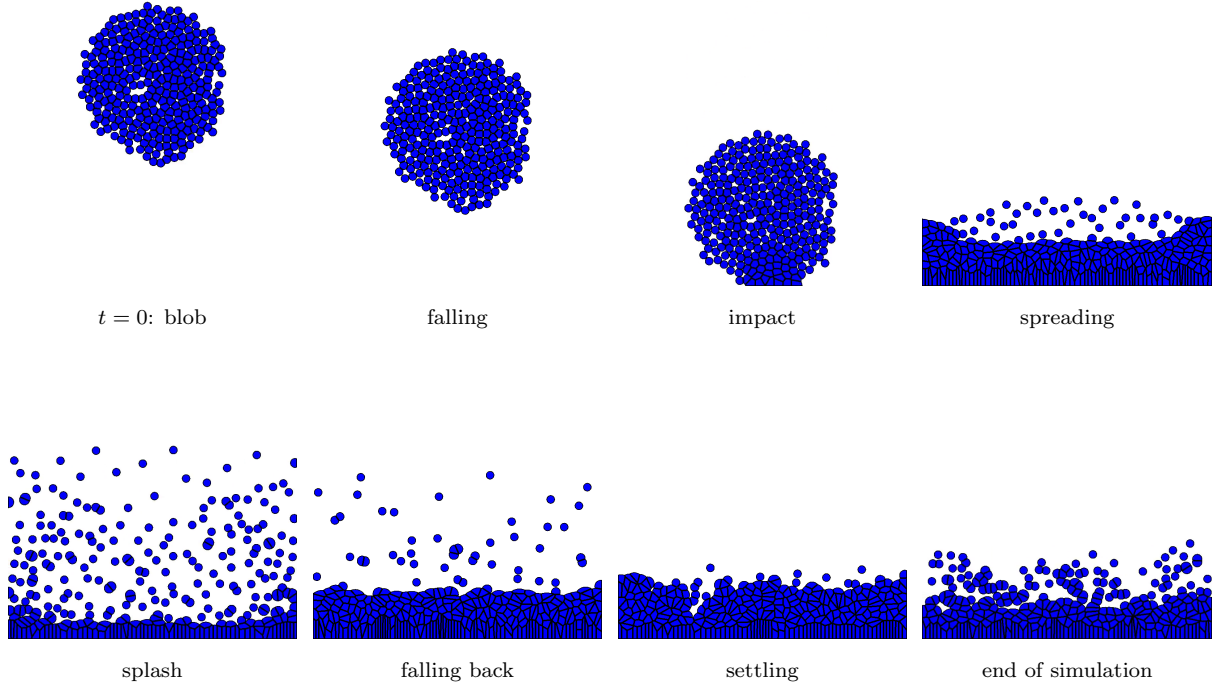


Figure 3: The 300-particle simulation over its full arc

A quick note on the behaviour: with $\varepsilon = 0.004$ the spring is very stiff, so the impact is energetic and the fluid scatters before settling rather than spreading smoothly. It is stable at $dt = 0.002$, but a calmer, more water-like result would most likely come from a larger ε or a smaller dt .

5 Validation

Beyond the visual results, I checked the building blocks against values whose answer is known in advance.

1. For the unit square the routines return `area` = 1, `centroid` = (0.5,0.5), and $\int_{\square} \|x - (0.5, 0.5)\|^2 dx = 0.16667$, which matches the closed-form value $1/6$ exactly, so `area`, `centroid` and `integral_square_distance` are correct ✓

2. For the 100-site diagram of Figure 1, the cell areas sum to 1.0000000 – the cells tile the whole unit square with no gaps or overlaps ✓
3. Running L-BFGS on 100 sites with a uniform target area $1/100$ brings the maximum per-cell area error down to 1.46×10^{-6} , so every cell reaches its target ✓
4. For the $N = 300$ fluid with target `fluid_volume` = 0.2, the total fluid area after the initial solve is 0.199992 (target 0.200000), with a maximum per-cell area error of 7.17×10^{-6} against the per-cell target $\approx 6.7 \times 10^{-4}$ ✓

6 Performance

The full 1000-frame fluid simulation at $N = 300$ runs in about 35 minutes on my i7-12700H (roughly 2s per frame). Almost all of that time is the optimal transport solve performed at every frame: L-BFGS rebuilds the whole diagram once per iteration, so the cost of a solve is the number of iterations times the cost of one rebuild – and both grow with N . Table 2 and Figure 4 show a standalone power-diagram solve (uniform target areas) swept from 100 to 5000 sites to test the per-frame cost of the fluid.

Sites N	Solve time	Iterations	Time / iteration
100	0.109 s	100	1.1 ms
200	0.389 s	151	2.6 ms
500	3.142 s	223	14.1 ms
1000	19.423 s	337	57.6 ms
2000	4 min 16 s	907	282 ms
5000	28 min 49 s	999*	1.73 s

Table 2: Cost of one full power-diagram + L-BFGS solve vs. number of sites

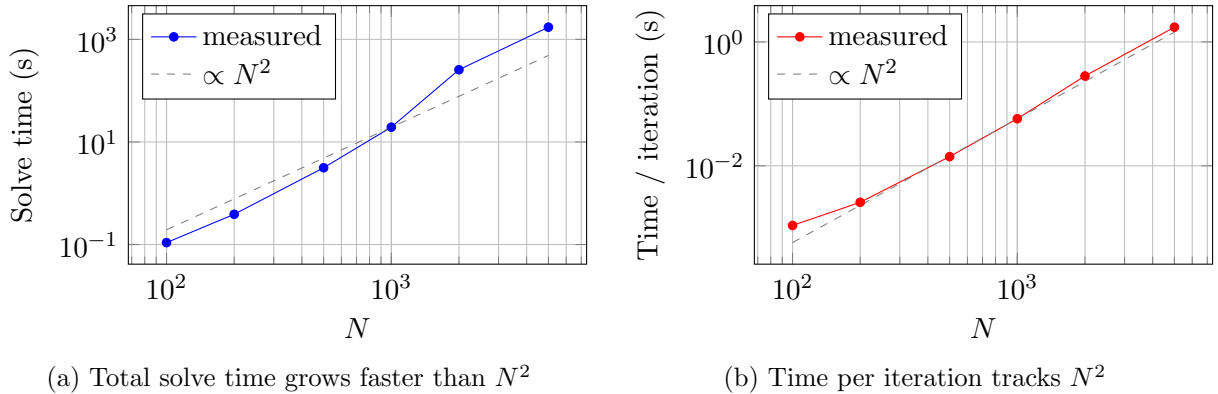


Figure 4: Scaling of the semi-discrete optimal transport solve (both axes are logarithmic)

Each L-BFGS iteration is one rebuild of the whole diagram, plus a cheap $\mathcal{O}(N)$ pass to read off the cell areas and integrals, so the time per iteration in Table 2 is essentially the cost of a single $\mathcal{O}(N^2)$ build – and indeed it tracks N^2 in Figure 4(b). The total solve time grows faster still (close to $N^{2.5}$) because the iteration count also rises with N . This is why we only use $N = 300$: more particles would give a smoother free surface, but at, say, $N = 2000$ a single frame already costs minutes, which would push the 1000-frame run from half an hour into days.

7 What is left to do

The solver is now complete and correct, but a few things could be improved with more time.

1. **Parallelise the diagram construction.** In the current code, `compute` runs serially (only the image rasteriser uses OpenMP). Since this construction is the bottleneck (it dominates the 35-minute run), parallelising it would speed up the whole simulation by roughly the core count. The only subtlety is that we must not `push_back` into the shared `cells` vector from several threads at once; instead we size it up front and have each thread write to its own slot:

```
void compute() {
    cells.resize(points.size());

    #pragma omp parallel for schedule(dynamic)
    for (int i = 0; i < (int)points.size(); i++) {
        Polygon cell;
        cell.vertices = {Vector(0,0), Vector(1,0), Vector(1,1), Vector(0,1)};

        for (int j = 0; j < (int)points.size(); j++) {
            if (i == j) continue;
            cell = clip_by_bisector(cell, points[i], points[j], weights[i], weights[j]);
        }
        // ... (the disk clipping for the fluid, unchanged) ...

        cells[i] = cell;
    }
}
```

2. **A k -d tree for nearest-neighbour clipping.** Even parallelised, the construction is $\mathcal{O}(n^2)$ because each cell is clipped against every other site. A k -d tree over the sites would let us clip only against nearby ones and stop early, bringing the cost down towards $\mathcal{O}(n \log n)$ and making large particle counts practical.
3. **A Newton solver instead of L-BFGS.** We could even use a Newton solver, which usually converges in far fewer iterations and would make each frame cheaper.

8 Conclusion

In its final form, we simulate 300 fluid particles over 1000 frames in about 35 minutes, producing a blob that falls, splashes and settles while every cell basically holds its prescribed area.

The project rides almost entirely on a single routine: the Sutherland–Hodgman clipper from Lab 6 is reused unchanged for the power bisectors in Lab 7 and for the disk edges in Lab 8, and the free surface falls straight out of the disk radius with no separate surface to track.

Declaration: All of the implementation in this project is my own work, written during the lab sessions, but I used an AI assistant to help with the report to generate an initial skeleton based on the structure of my first project report and refine the wording.

9 Course feedback

Overall, I found the course very interesting, and I really enjoyed its practical approach. I generally find that learning through projects is a lot more efficient than the classic lectures + TD format. My main piece of feedback is that, for me, the main difficulty in the course was the C++ implementation rather than the theory and the algorithms themselves, which I think we could have spent more time on.

The first project felt very reasonable, but the second was more involved, so an extra lab session to work on it (notably on the optimisations) would have been welcome.

In any case, thank you for a course that I really enjoyed!